

# Damara FE UI 기준 백엔드 보완 요청서

작성 기준: 현재 프론트 UI 화면 기준

목적: 목데이터/임시 UI를 제거하고 실제 API 데이터로 모든 화면을 안정적으로 연결하기 위함

## 0. 현재 상황 요약

현재 Swagger 기준으로 `Posts`, `Users`, `Chat`, `Notifications`, `Upload` API는 기본 기능이 존재합니다. 다만 프론트 화면은 홈, 카테고리, 공구 상세, 공구 등록, 마이페이지, 설정, 공지사항, FAQ, 채팅까지 확장되어 있어서 현재 API만으로는 일부 UI를 실제 데이터로 채우기 어렵습니다.

특히 아래 부분이 부족합니다.

- 목록 응답 형태가 화면마다 일관되게 쓰기 어렵습니다.
- 상세 화면에 필요한 판매자 정보, 참여자 정보, 수령 안내, 거래 유의사항 필드가 부족합니다.
- 홈/카테고리 화면의 검색, 정렬, 필터, 짐 상태가 사용자 기준으로 안정적으로 내려와야 합니다.
- 마이페이지는 여러 API를 조합해야 해서 summary API가 필요합니다.
- 공지사항, FAQ, 설정 화면은 현재 프론트 임시 데이터입니다.
- 채팅은 REST API는 있으나 실시간 UX에는 WebSocket 또는 SSE가 필요합니다.

## 1. 공통 응답 규칙 정리 요청

### 문제

Swagger 예시에서는 `/api/posts` 가 배열로 내려오지만, 실제 호출에서는 `{ value: [...] }` 형태로 내려오는 경우가 있습니다. 프론트에서 매번 `posts`, `items`, `data`, `value` 를 모두 대응해야 합니다.

### 요청

목록 API 응답 형태를 통일해 주세요.

추천 형태:

```
{
  "items": [],
  "total": 0,
  "limit": 20,
  "offset": 0,
  "hasNext": false
}
```

또는 도메인명을 쓰는 경우:

```
{
  "posts": [],
  "total": 0,
  "limit": 20,
  "offset": 0,
  "hasNext": false
}
```

프론트 입장에서는 모든 목록 API가 같은 패턴이면 가장 좋습니다.

- `/api/posts`
- `/api/chat/rooms/user/{userId}`
- `/api/notifications`
- `/api/users/{id}/trust-events`
- `/api/notices`
- `/api/faqs`

---

## 2. 홈/카테고리 게시물 목록 API 보완

관련 화면:

- 홈 화면
- 카테고리 화면
- 검색 결과
- 정렬 탭
- 찜 버튼
- 공구 상세 진입

현재 Swagger:

`GET /api/posts`

현재 지원:

- `limit`
- `offset`
- `category`
- `sort`
- `status`
- `keyword`
- `q`
- `userId`
- `x-user-id`

### 추가/확정 요청

#### 2.1 응답에 사용자 기준 상태 포함

로그인 사용자를 넘겼을 때 각 게시물에 아래 값이 반드시 포함되면 좋습니다.

```
{
  "isFavorite": true,
  "isParticipant": false,
  "isOwner": false
}
```

필요 이유:

- 카드 찜 버튼 초기 상태 표시
- 상세 화면 참여하기/참여취소 버튼 분기

- 작성자 본인 글인 경우 수정/삭제/상태변경 UI 노출

## 2.2 카드 UI용 필드 추가

카드에서 필요한 최소 필드:

```
{
  "id": "uuid",
  "title": "물티슈 공동구매",
  "price": 5900,
  "currentQuantity": 1,
  "minParticipants": 3,
  "status": "open",
  "category": "daily",
  "pickupLocation": "명지대 정문앞",
  "deadline": "2026-05-20T18:00:00.000Z",
  "images": [
    {
      "id": "uuid",
      "imageUrl": "https://...",
      "sortOrder": 0
    }
  ],
  "thumbnailUrl": "https://...",
  "favoriteCount": 12,
  "isFavorite": true,
  "isParticipant": false,
  "isOwner": false,
  "createdAt": "2026-05-15T00:00:00.000Z"
}
```

`thumbnailUrl` 은 필수는 아니지만 있으면 프론트가 첫 번째 이미지를 직접 계산하지 않아도 됩니다.

## 2.3 마감 상태 계산 필드

프론트에서 날짜 계산을 할 수도 있지만, 서버 기준 시간이 더 정확합니다.

추가 희망:

```
{
  "deadlineStatus": "open",
  "deadlineLabel": "오늘 마감",
  "remainingSeconds": 3600
}
```

가능 값:

- `open`
- `closingSoon`
- `closed`

필요 이유:

- 카드 배지: `모집중`, `마감임박`, `마감`
- 정렬: 마감임박순
- 홈 인기/마감 섹션 구성

### 3. 카테고리 값 enum 확정

현재 Swagger 기준:

- `food` : 먹거리
- `daily` : 생활용품
- `beauty` : 뷰티·패션
- `electronics` : 전자기기
- `school` : 학용품
- `freemarket` : 프리마켓

#### 요청

등록, 수정, 목록, 상세 응답 모두 위 enum으로 통일해 주세요.

현재 프론트에서 과거 값과 함께 대응 중인 값:

- `stationery`
- 생활용품
- 먹거리
- 뷰티·패션
- 학용품

앞으로는 서버/프론트 모두 Swagger enum만 사용하면 좋습니다.

### 4. 검색/필터/정렬 API 동작 명확화

관련 화면:

- 홈 최신순/마감임박순/인기순
- 카테고리 검색
- 카테고리 chip 필터

#### 요청

`GET /api/posts` 에서 아래 조합이 모두 동작해야 합니다.

```
GET /api/posts?status=open&category=daily&sort=popular&q=물티슈&limit=20&offset=0&userId={userId}
```

정렬 기준 정의 요청:

- `latest` : `createdAt DESC`
- `deadline` : 마감 전 게시글 우선, `deadline ASC`
- `popular` : `currentQuantity DESC` 또는 `favoriteCount DESC` 기준 중 하나 확정 필요

추천:

```
popular = currentQuantity DESC, favoriteCount DESC, createdAt DESC
```

검색 대상:

- `title`
  - `content`
  - `pickupLocation`
  - 가능하면 `category` label도 포함
- 

## 5. 게시글 상세 API 보완

관련 화면:

- `/post/{id}` 공구 상세 화면

현재 상세 UI에는 아래 섹션이 있습니다.

- 대표 이미지/이미지 캐러셀
- 제목, 가격, 카테고리, 인기 배지
- 수령 장소
- 마감일
- 모집 인원 진행률
- 예상 수령일
- 판매자 정보
- 참여자 정보
- 상품 소개
- 수령 안내
- 거래 유의사항
- 채팅 버튼
- 참여하기/참여취소 버튼
- 찜 버튼

### 요청

`GET /api/posts/{id}` 응답에 아래 구조를 포함해 주세요.

```
{
  "id": "uuid",
  "authorId": "uuid",
  "title": "물티슈 공동구매",
  "productName": "다마라 물티슈",
  "content": "상품 설명",
  "price": 5900,
  "minParticipants": 3,
  "currentQuantity": 1,
  "status": "open",
  "deadline": "2026-05-20T18:00:00.000Z",
  "pickupLocation": "명지대 정문앞",
  "pickupDate": "2026-05-21",
  "pickupStartTime": "12:00",
  "pickupEndTime": "18:00",
  "pickupGuide": "정문 앞에서 채팅 후 수령",
  "category": "daily",
  "groupBuyType": "PRE_RECRUIT",
  "images": [
    {
      "id": "uuid",
      "imageUrl": "https://...",
      "sortOrder": 0
    }
  ],
  "tags": ["인기", "생활용품"],
  "notice": "마감 후 취소가 어려울 수 있어요.",
  "favoriteCount": 12,
  "isFavorite": true,
  "isParticipant": false,
  "isOwner": false,
  "deadlineStatus": "closingSoon",
  "deadlineLabel": "오늘 마감",
  "author": {
    "id": "uuid",
    "nickname": "노승민",
    "studentId": "12341234",
    "department": "컴퓨터공학과",
    "avatarUrl": "https://...",
    "trustScore": 86,
    "trustGrade": 4.3,
    "responseTimeLabel": "10분 이내",
    "completedTradeRate": 98,
    "badges": ["꼼꼼해요", "친절해요", "약속시간 잘 지켜요"]
  },
  "participantsPreview": [
    {
      "userId": "uuid",
      "nickname": "참여자 1",
      "avatarUrl": "https://...",

```

```
    "trustGrade": 4.1,
    "joinedAt": "2026-05-15T00:00:00.000Z"
  }
],
"participantsTotal": 2
}
```

## 6. 공구 등록 API 보완

관련 화면:

- [/create](#) 공구 등록 5단계 UI

현재 등록 UI 입력값:

1. 상품 이미지, 상품명, 공구 제목
2. 공구 방식, 카테고리
3. 가격, 모집 인원, 현재 참여 인원
4. 수령 장소, 마감일, 예상 수령일
5. 상세 설명, 태그, 등록 전 확인

### 요청

[POST /api/posts](#) 에서 아래 필드를 받을 수 있게 해주세요.

```
{
  "post": {
    "authorId": "uuid",
    "title": "물티슈 공동구매",
    "productName": "다마라 물티슈",
    "content": "상세 설명",
    "price": 5900,
    "minParticipants": 3,
    "deadline": "2026-05-20T18:00:00.000Z",
    "pickupLocation": "명지대 정문앞",
    "pickupDate": "2026-05-21",
    "pickupStartTime": "12:00",
    "pickupEndTime": "18:00",
    "pickupGuide": "정문 앞에서 채팅 후 수령",
    "category": "daily",
    "groupBuyType": "PRE_RECRUIT",
    "images": ["https://..."],
    "tags": ["생활용품", "인기예상"],
    "notice": "마감 후 취소가 어려울 수 있어요."
  }
}
```

[currentQuantity](#) 는 보통 서버에서 0으로 시작하는 것이 안전합니다. 다만 디자인/테스트용으로 현재 참여 인원을 넣을 수 있어야 한다면 관리자/개발용 필드로만 허용하는 것을 추천합니다.

## 7. 게시물 수정/삭제/상태 변경 권한

관련 화면:

- 내가 올린 공구
- 공구 상세 작성자 메뉴
- 마이페이지 등록 공구 관리

### 요청

GET /api/posts/{id} 에 isOwner 포함:

```
{
  "isOwner": true
}
```

PATCH /api/posts/{id}/status 상태 전이 규칙 명확화:

가능 상태:

- open
- closed
- in\_progress
- completed
- cancelled

추천 전이:

- open -> closed
- open -> cancelled
- closed -> in\_progress
- in\_progress -> completed
- in\_progress -> cancelled

프론트가 에러 메시지를 자연스럽게 보여줄 수 있도록 에러 코드도 고정 부탁드립니다.

```
{
  "error": "INVALID_STATUS_TRANSITION",
  "message": "현재 상태에서는 완료 처리할 수 없습니다."
}
```

## 8. 참여자 API 보완

관련 화면:

- 공구 상세 참여자 섹션
- 참여 현황 보기
- 채팅에서 참여자 정보 표시

현재:

GET /api/posts/{id}/participants



## 요청 응답

```
{
  "participants": [
    {
      "userId": "uuid",
      "nickname": "참여자 1",
      "studentId": "20241234",
      "department": "컴퓨터공학과",
      "avatarUrl": "https://...",
      "trustGrade": 4.1,
      "joinedAt": "2026-05-15T00:00:00.000Z",
      "status": "joined"
    }
  ],
  "total": 2,
  "limit": 20,
  "offset": 0
}
```

참여 상태 값:

- `joined`
- `cancelled`
- `completed`

## 9. 참여/참여취소 응답 개선

관련 화면:

- 상세 하단 `참여하기`
- 상세 하단 `참여취소`
- 모집 인원 progress bar

현재는 참여 후 프론트가 수량을 임시로 계산해야 합니다.

## 요청

`POST /api/posts/{id}/participate`

응답:

```
{
  "isParticipant": true,
  "post": {
    "id": "uuid",
    "currentQuantity": 2,
    "minParticipants": 3,
    "status": "open"
  }
}
```

DELETE /api/posts/{id}/participate/{userId}

응답:

```
{
  "isParticipant": false,
  "post": {
    "id": "uuid",
    "currentQuantity": 1,
    "minParticipants": 3,
    "status": "open"
  }
}
```

에러 코드:

- ALREADY\_PARTICIPATED
- OWNER\_CANNOT\_PARTICIPATE
- POST\_CLOSED
- POST\_NOT\_FOUND
- USER\_NOT\_FOUND

---

## 10. 찜 API 보완

관련 화면:

- 홈 카드
- 카테고리 카드
- 상세 상단 하트
- 관심목록 페이지

현재 API는 존재합니다.

- POST /api/posts/{postId}/favorite
- GET /api/posts/{postId}/favorite/{userId}
- DELETE /api/posts/{postId}/favorite/{userId}
- GET /api/users/{userId}/favorites

### 요청

목록/상세 응답에 `isFavorite`, `favoriteCount` 가 항상 포함되면 개별 카드마다 `checkFavorite` 를 추가 호출하지 않아도 됩니다.

찜 등록/해제 응답도 최신 count 포함:

```
{
  "isFavorite": true,
  "favoriteCount": 13
}
```

관심목록 응답은 아래 형태 추천:

```
{
  "favorites": [
    {
      "id": "favoriteId",
      "postId": "postId",
      "post": {}
    }
  ],
  "total": 10,
  "limit": 20,
  "offset": 0
}
```

---

## 11. 마이페이지 summary API 추가

관련 화면:

- 마이페이지 상단 카운트
- 프로필 신뢰 점수
- 하단 탭 채팅/알림 배지

현재 프론트는 여러 API를 동시에 호출해야 합니다.

### 추가 요청

GET /api/users/{id}/summary

응답:

```
{
  "user": {
    "id": "uuid",
    "nickname": "노승민",
    "studentId": "12341234",
    "department": "컴퓨터공학과",
    "avatarUrl": "https://...",
    "trustScore": 86,
    "trustGrade": 4.5
  },
  "counts": {
    "createdPostCount": 5,
    "participatedPostCount": 2,
    "favoriteCount": 8,
    "unreadChatCount": 3,
    "unreadNotificationCount": 4
  },
  "trust": {
    "label": "신뢰도 좋은 거래 파트너예요",
    "badges": ["꼼꼼해요", "친절해요", "약속시간 잘 지켜요"],
    "completedTradeCount": 12,
    "responseRate": 92,
    "cancelCount": 1,
    "noShowCount": 0
  }
}
```

필요 이유:

- 마이페이지 첫 렌더링 속도 개선
- API 호출 수 감소
- 신뢰도 카드 실제 데이터 연결

## 12. 신뢰도/매너 점수 API 보완

관련 화면:

- 마이페이지 프로필 카드
- 안전거래 프로필
- 공구 상세 판매자 카드
- 참여자 카드

현재:

`GET /api/users/{id}/trust-events`

### 추가 요청

이벤트 이력과 별도로 현재 요약값 API가 필요합니다.

`GET /api/users/{id}/trust-summary`

응답:

```
{
  "trustScore": 86,
  "trustGrade": 4.5,
  "gradeLabel": "매너 학생",
  "rankPercent": 23,
  "completedTradeCount": 12,
  "responseRate": 92,
  "avgResponseMinutes": 10,
  "cancelCount": 1,
  "noShowCount": 0,
  "badges": ["꼼꼼해요", "친절해요", "약속시간 잘 지켜요"]
}
```

## 13. 공지사항 API 추가

관련 화면:

- 마이페이지 > 공지사항

현재는 프론트 임시 데이터입니다.

### 추가 요청

GET /api/notices

```
{
  "notices": [
    {
      "id": "uuid",
      "title": "Damara 베타 서비스 오픈",
      "summary": "명지대 캠퍼스 공동구매를 더 쉽게 이용할 수 있어요.",
      "content": "상세 내용",
      "type": "service",
      "isPinned": true,
      "createdAt": "2026-05-15T00:00:00.000Z"
    }
  ],
  "total": 1
}
```

GET /api/notices/{id}

상세 내용 응답.

공지 타입:

- service
- event
- maintenance
- policy

## 14. FAQ API 추가

관련 화면:

- 마이페이지 > FAQ

현재는 프론트 임시 데이터입니다.

### 추가 요청

GET /api/faqs

```
{
  "faqs": [
    {
      "id": "uuid",
      "category": "trade",
      "question": "공구 참여는 어떻게 하나요?",
      "answer": "공구 상세 화면에서 참여하기를 누르면 돼요.",
      "order": 1,
      "isActive": true
    }
  ]
}
```

카테고리:

- trade
- account
- payment
- pickup
- etc

---

## 15. 사용자 설정 API 추가

관련 화면:

- 마이페이지 > 설정

현재 설정 토글은 프론트 상태만 바뀝니다.

### 추가 요청

GET /api/users/{id}/settings

```
{
  "pushEnabled": true,
  "chatNotificationEnabled": true,
  "postNotificationEnabled": true,
  "marketingNotificationEnabled": false,
  "quietHoursEnabled": false,
  "quietHoursStart": "23:00",
  "quietHoursEnd": "08:00"
}
```

PUT /api/users/{id}/settings

동일 body로 저장.

## 16. 알림 API 보완

관련 화면:

- 홈 상단 알림 아이콘
- 알림 목록 페이지 예정
- 하단/상단 unread badge

현재 알림 API는 존재합니다.

### 요청

알림 타입 enum 확정:

- new\_participant
- post\_deadline\_soon
- post\_closed
- post\_status\_changed
- new\_chat\_message
- favorite\_post\_deadline\_soon
- trade\_completed
- trade\_cancelled
- system\_notice

알림 응답에 action target 추가:

```
{
  "id": "uuid",
  "type": "new_participant",
  "title": "새로운 참여자",
  "message": "물티슈 공동구매에 새로운 참여자가 있어요.",
  "postId": "uuid",
  "chatRoomId": "uuid",
  "actionUrl": "/post/{id}",
  "isRead": false,
  "createdAt": "..."
}
```

필요 이유:

- 알림 클릭 시 상세/채팅으로 이동
  - 알림 종류별 아이콘/색상 매칭
- 

## 17. 채팅 목록 API 보완

관련 화면:

- 채팅 탭
- 상세에서 채팅 버튼 클릭 후 채팅방 자동 진입

현재:

- `GET /api/chat/rooms/user/{userId}`
- `GET /api/chat/rooms/post/{postId}`

### 요청

채팅방 목록 응답에 UI에 필요한 값 포함:



```
{
  "chatRooms": [
    {
      "id": "uuid",
      "postId": "uuid",
      "post": {
        "id": "uuid",
        "title": "물티슈 공동구매",
        "status": "open",
        "pickupLocation": "명지대 정문앞",
        "deadline": "2026-05-20T18:00:00.000Z",
        "thumbnailUrl": "https://...",
        "authorId": "uuid"
      },
      "participants": [
        {
          "userId": "uuid",
          "nickname": "노승민",
          "avatarUrl": "https://..."
        }
      ],
      "lastMessage": {
        "id": "uuid",
        "content": "오늘 오후 5시까지 수령 가능해요.",
        "senderId": "uuid",
        "messageType": "text",
        "createdAt": "..."
      },
      "unreadCount": 2,
      "updatedAt": "..."
    }
  ],
  "total": 0,
  "limit": 20,
  "offset": 0
}
```

## 18. 채팅 메시지 API 보완

관련 화면:

- 채팅 상세 오버레이
- 시스템 메시지
- 이미지 메시지 예정

현재:

- [GET /api/chat/rooms/{chatRoomId}/messages](#)
- [POST /api/chat/messages](#)

## 요청

메시지 응답:

```
{
  "messages": [
    {
      "id": "uuid",
      "chatRoomId": "uuid",
      "senderId": "uuid",
      "sender": {
        "id": "uuid",
        "nickname": "노승민",
        "avatarUrl": "https://..."
      },
      "content": "안녕하세요.",
      "messageType": "text",
      "isRead": true,
      "createdAt": "..."
    }
  ],
  "total": 0,
  "limit": 50,
  "offset": 0
}
```

메시지 타입:

- `text`
- `image`
- `system`

시스템 메시지 예:

```
{
  "messageType": "system",
  "content": "공구 참여가 확정됐어요."
}
```

---

## 19. 채팅 실시간 기능 요청

REST API만으로는 채팅 UX가 부족합니다.

### 추가 요청

WebSocket 또는 SSE 중 하나 필요.

추천 WebSocket 이벤트:

클라이언트 -> 서버:

- `chat:join`
- `chat:leave`

- `chat:send`
- `chat:read`

서버 -> 클라이언트:

- `chat:message`
- `chat:read`
- `chat:roomUpdated`
- `notification:new`

필요 이유:

- 채팅방에서 메시지 즉시 반영
- 채팅 목록의 `lastMessage` 갱신
- `unreadCount` 실시간 갱신
- 하단 탭/알림 아이콘 `badge` 실시간 갱신

---

## 20. 이미지 업로드/응답 형태 통일

현재 Swagger에는 단일/다중 업로드가 있습니다.

- `POST /api/upload/image`
- `POST /api/upload/images`

### 요청

게시글 응답의 `images` 형태를 전체 API에서 통일해 주세요.

추천:

```
{
  "images": [
    {
      "id": "uuid",
      "imageUrl": "https://damara.bluerack.org/uploads/images/abc.png",
      "sortOrder": 0
    }
  ]
}
```

그리고 업로드 응답도 가능하면 같은 구조면 좋습니다.

```
{
  "images": [
    {
      "imageUrl": "/uploads/images/abc.png",
      "sortOrder": 0
    }
  ]
}
```

---

## 21. 에러 응답 포맷 통일

프론트에서 토스트/하단시트로 에러를 보여주려면 에러 포맷이 일정해야 합니다.

### 요청

모든 API 에러 응답:

```
{
  "error": "POST_CLOSED",
  "message": "마감된 공구에는 참여할 수 없습니다.",
  "details": {}
}
```

대표 에러 코드:

- `VALIDATION_ERROR`
- `UNAUTHORIZED`
- `FORBIDDEN`
- `NOT_FOUND`
- `POST_NOT_FOUND`
- `USER_NOT_FOUND`
- `POST_CLOSED`
- `ALREADY_PARTICIPATED`
- `ALREADY_FAVORITED`
- `INVALID_STATUS_TRANSITION`
- `UPLOAD_FILE_TOO_LARGE`

---

## 22. 프론트 우선순위 기준 요청

### 1순위: 실제 데이터 연결 필수

- `/api/posts` 응답 형태 통일
- 목록/상세에 `isFavorite`, `favoriteCount`, `isParticipant`, `isOwner` 포함
- 이미지 응답 형태 통일
- 카테고리 enum 통일
- 검색/필터/정렬 조합 안정화

### 2순위: 상세/참여 UX 완성

- 상세 `author` 포함
- 상세 `participantsPreview` 포함
- 참여/참여취소 응답에 최신 post 수량 반환
- 판매자 trust summary 제공

### 3순위: 마이페이지/서비스 화면 실재화

- `/api/users/{id}/summary`
- `/api/users/{id}/trust-summary`
- `/api/notices`

- `/api/faqs`
- `/api/users/{id}/settings`

#### 4순위: 실시간 UX

- 채팅 WebSocket/SSE
- 알림 실시간 이벤트
- unread count 실시간 갱신

## 23. 프론트에서 바로 맞출 수 있는 이상적인 API 호출 예시

### 홈 최신순

```
GET /api/posts?status=open&sort=latest&limit=20&offset=0&userId={userId}
```

### 카테고리 생활용품

```
GET /api/posts?status=open&category=daily&sort=latest&limit=20&offset=0&userId={userId}
```

### 검색

```
GET /api/posts?q=물티슈&category=daily&sort=popular&limit=20&offset=0&userId={userId}
```

### 상세

```
GET /api/posts/{postId}?userId={userId}
```

### 마이페이지

```
GET /api/users/{userId}/summary
```

### 채팅 목록

```
GET /api/chat/rooms/user/{userId}?limit=20&offset=0
```

### 알림 unread

```
GET /api/notifications/unread-count?userId={userId}
```

## 24. 최종 요약

현재 UI를 실제 서비스처럼 완성하려면 백엔드에서 가장 먼저 아래 5가지를 맞춰주면 됩니다.

1. `GET /api/posts` 목록 응답 통일 및 `isFavorite/isParticipant/isOwner` 포함
2. `GET /api/posts/{id}` 상세 응답에 `author`, `participantsPreview`, 수령 안내 필드 포함
3. `POST/DELETE participate` 응답에 최신 `currentQuantity/status` 반환

4. `GET /api/users/{id}/summary` 추가

5. 공지사항/FAQ/설정 API 추가

이후 채팅 WebSocket/SSE와 알림 자동 생성까지 붙이면 현재 프론트 UI 기준으로 목데이터 없이 대부분의 화면을 실제 데이터로 운영할 수 있습니다.